aerrand

Building the “USB Port” for Secondhand Marketplaces: API Key Infrastructure for Trust and Logistics

Project proposal for the University of Windsor Master of Applied Computing Industry Partnership Program

This document is a plain-language overview. A full technical requirements document, covering the data model, API specification, security requirements, and implementation plan, is included separately for teams who want to go deeper before proposing their approach.

About Aerrand

Aerrand is a Windsor based startup making it safe to buy and sell secondhand items on platforms like Kijiji and Facebook Marketplace, without ever meeting a stranger. We connect buyers to vetted local drivers (“Aerranders”) who verify, pick up, and deliver the item. We're live in Windsor with real deliveries happening today, and we're building toward becoming the trust and logistics layer embedded directly inside marketplace platforms.

This is not a hypothetical case study. Whatever the winning team builds ships to a production system that real businesses and drivers use.

The Problem, in One Sentence

Right now, a business that wants Aerrand to deliver something has to log into our dashboard and click through a form by hand. We want them to be able to do that from their own software instead, automatically, the same way a checkout page can say “Log in with Google” instead of building its own login system. That handshake, a business's software proving “I am Acme Co. and I'm allowed to book,” happens through an API key.

How This Fits Into What We Have Today

Aerrand's backend is a Django API paired with a Node.js service that talks to the driver app, with MongoDB behind both. Today, when the dashboard creates a delivery, the price comes from the browser and the server trusts it, which is fine when our own dashboard is the only caller. It stops being fine the moment any outside program can call us with a key, so the server has to calculate the price itself. We're also not bolting keys onto the existing endpoints, since many of those were built for our own dashboard and aren't locked down for outside traffic. Instead, students would build a new, separate front door, `api.aerrand.com/v1`, that reuses our existing pricing and dispatch logic underneath but exposes nothing else. New entrance, not a new building.

A Concrete Example

Imagine Kijiji wants to offer “verified secondhand delivery” as a checkout option on their own site, without building any verification or driver logistics themselves. With an Aerrand API key, their engineering team calls a single endpoint, something like `POST /v1/aerrands`, passes in the listing and buyer info, and Aerrand handles the rest: pricing it, dispatching a driver, verifying the item, and updating them the moment it's delivered. The student project is what makes that handshake possible: secure, priced correctly, trackable, and easy enough that a partner's dev team could integrate it in an afternoon.

What Students Would Actually Build

1. Key issuance and authentication. A system that generates API keys, stores only a one-way hash of each key (never the key itself), and authenticates every incoming request against it. Keys carry scopes (create, read, cancel) and can be revoked instantly if one leaks.

aerrand

2. A priced, server-trusted booking endpoint. The server calculates the price itself, using our existing pricing engine, and never accepts a price from the caller. Students would build `POST /v1/quotes` (price it before booking), `POST /v1/aerrands` (create a delivery, correctly priced, with multi-stop support), `GET /v1/aerrands/{id}` and its tracking endpoint, and `POST /v1/aerrands/{id}/cancel`, which reuses our existing cancellation fee rules.

3. Automated payment. Businesses already have a prepaid wallet on Aerrand. Right now nothing uses it automatically. Students would wire it up so a booking through the API can pay for itself instantly (subtract from the balance, or fail cleanly if funds are short), no human required to click “pay” on a Stripe page.

4. Webhooks. Instead of a partner's system having to constantly ask “is it done yet?”, we push events to them the moment something happens: `aerrand.created`, `paid`, `accepted`, `picked_up`, `delivered`, `cancelled`. The tricky part: driver actions happen in the Node service, which the Django API never hears about directly. Students would design a small “outbox” table that both services write to when something happens, and a background dispatcher that reads it, signs each event, and delivers it to the partner's URL, retrying if their server is down.

5. Safety rails that show up in any real API. *Idempotency:* if a network call is retried, a client-supplied idempotency key ensures we don't accidentally create the same delivery twice. *Rate limiting:* each key gets a request budget, backed by Redis, so one partner can't overload the system. *Tenant isolation:* every query is scoped to the business that owns the key, so a business can never see or cancel another business's aerrand, even if it guesses the ID.

What “Done” Looks Like

By the end, a business should be able to do all of this without ever talking to us directly: get approved and open a Developers section in the dashboard, create a test key and build against the docs, register a webhook URL and watch events arrive, switch to a live key and create real deliveries that show up in the driver app, and cancel, track, and reconcile everything from their own system. On our side: prices can never be tampered with, keys are never stored in plaintext, and a leaked key can be revoked instantly with zero impact on the existing dashboard or driver experience.

Why This Is a Strong Team Project

- **It's real, scoped infrastructure**, not a toy assignment: authentication, pricing integrity, payments, and event delivery are all things production APIs have to get right, and getting them right is a genuinely interesting systems problem.
- **The output is directly usable.** This maps straight onto conversations we're already having with real marketplace partners, so a strong submission doesn't sit on a shelf.
- **Clear success criteria for judging:** by the end, a business should be able to get approved, generate a test key, read the docs, and create a working test delivery in under 10 minutes, entirely on their own, no help from us. That's an easy, concrete bar for comparing teams.

Prepared by Johnpaul Sani, Founder and CEO, Aerrand Technologies Inc.